7_26_marzo

Tra i Thread utente che fanno riferimento ad un unico processo, modello *1 a molti*, i mutex si possono usare in modo efficiente facendo uso di TSL (o XCHG)

Mutex e thread utente

Tra i Thread utente che fanno riferimento ad un unico processo, modello *1 a molti*, i mutex si possono usare in modo efficiente con TSL (o XCHG)

> **Cite**
>
> **Ricordiamo**:
>
> - **TSL (Test and Set Lock):** È un'istruzione hardware che esegue due operazioni contemporaneamente e in modo indivisibile: **legge** il valore di una locazione di memoria (il "lock") copiandolo in un registro della CPU, e subito dopo **imposta** (scrive) quella stessa locazione di memoria a un valore non nullo (solitamente `1`, per indicare che la risorsa è "occupata").
> - *A cosa serve:* Se il valore letto era `0`, il processo capisce che il lucchetto era libero, lo ha appena chiuso a chiave (`1`) ed entra nella sua sezione critica. Se il valore letto era `1`, sa che qualcun altro è già dentro e continua a ciclare (Spinlock) finché non si libera.
> - **XCHG (Exchange)** *(nota: spesso la troverai scritta `XCHG` invece di `XHCG`, tipica delle CPU x86)*: È un'istruzione hardware alternativa al TSL che **scambia** (exchange) in modo atomico il contenuto di un registro della CPU con il contenuto di una locazione di memoria.
> - *A cosa serve:* Il meccanismo è simile al TSL. Un processo mette il valore `1` in un suo registro interno e poi esegue la `XCHG` con la variabile di lock in memoria. Dopo lo scambio, il processo controlla il suo registro: se c'è uno `0`, significa che ha "rubato" lo zero alla memoria lasciandoci un `1` (ha acquisito il lock). Se c'è un `1`, il lock era già preso e deve riprovare.
>
> ```
> enter_region:
>  TSL REGISTER,LOCK
>  CMP REGISTER,#0
>  JNE enter_region
>  RET
> ```
>
> ```
> leave_region:
>  MOVE LOCK,#0
>  RET
> ```

L'implementazione di queste due istruzioni cambia e andremo ad usare delle primitive introdotte precedentemente (thread_yield)

```
mutex_lock:
    TSL REGISTER,MUTEX
    CMP REGISTER,#0
    JZE ok
    CALL thread_yield
    JMP mutex_lock
ok:RET
```

```
mutex_unlock:
    MOVE MUTEX,#0
    RET
```

FUTEX (Fast Userspace Mutex)
Garantiscono prestazioni eccellenti riducendo drasticamente le costose chiamate di sistema (*system call*). L'idea chiave è tentare di gestire il lock interamente in modalità utente. Si compongono di due elementi:

- **Libreria utente:** Tenta di acquisire il lock usando istruzioni hardware atomiche (come le TSL o XCHG viste prima). Se il lock è libero, lo prende e ritorna istantaneamente. Non essendoci chiamate al sistema operativo, l'operazione è velocissima.
- **Servizio kernel:** Interviene **solo** in caso di contesa. Se la libreria utente trova il lock già occupato, effettua una chiamata a sistema per chiedere al kernel di bloccare il processo (mettendolo in *sleep*) e accodarlo finché la risorsa non si libera.

Monitor

Sono delle astrazioni di alto livello disponibili su alcuni linguaggi, che offrono una gestione semplice della mutua esclusione, che sollevano il programmatore della responsabilità di usare bene i mutex.

> **Tip**
>
> I semafori sono universali non essendo legate al linguaggio di programmazione, sono delle chiamate a sistema a differenza dei monitor che sono dei costrutti di più alto livello.

Quando i linguaggi di programmazione introducono i monitor molto spesso integrano anche una gestione autonoma dei thread, per garantire la sincronizzazione il monitor crea una variabile condizione sulla quale esercita i *wait* e *signal* (simili a sleep e wake up)

- fatta da Haore (so solo che non è stata mai implementata)
- Mesa: implementazione usata anche da Java
- concurrent Pascal: nessuna la usa, ma usa un approccio diverso

Di seguito la soluzione dei produttori-consumatori ma usando i monitor

```
monitor pc_monitor
   condition full, empty;
   integer count = 0;

   function insert(item)
      if count = N then wait(full);
      insert_item(item);
      count = count + 1;
      if count = 1 then signal(empty)


   function remove()
      if count = 0 then
         wait(empty);
      remove = remove_item()
      count = count - 1;
      if count = N-1 then signal(full)
```

```
function producer()
   while (true) do
      item = produce_item()
      pc_monitor.insert(item)
```

```
function consumer()
   while (true) do
      item = pc_monitor.remove()
      consume_item(item)
```

mario di raimondo

Scambio messaggi tra processi

Questo approccio utilizza due primitive ad alto livello fornite dal sistema operativo: `send(destination,message)` e `receive(source,message)`.
I concetti chiave da ricordare sono tre:

- **Sincronizzazione implicita (Blocchi):** La `receive` è tipicamente *bloccante*: se non ci sono messaggi da leggere, il processo si mette in attesa. Anche la `send` può esserlo: se assumiamo che ci sia un **buffer** di appoggio con capienza finita *N*, e questo buffer è pieno, chi spedisce si blocca finché non si libera spazio.
- **Indirizzamento:** Può essere **diretto** (il mittente specifica il PID esatto del processo destinatario) oppure indiretto tramite una **mailbox** (una "cassetta della posta" condivisa da cui i processi pescano i messaggi).
- **Scalabilità:** Il grande vantaggio di questo approccio è che non richiede memoria condivisa. Per questo motivo è facilmente estendibile a sistemi distribuiti, permettendo a processi su computer diversi di comunicare via rete (ad esempio usando le librerie **MPI**).

```
function producer()
    while (true) do
        item = produce_item()
        build_msg(m,item)
        send(consumer, msg)
```

```
function consumer()
    while (true) do
        receive(producer, msg)
        item=extract_msg(msg)
        consum_item(item)
```

Il Codice (Produttore-Consumatore): risolvere il problema del Bounded-Buffer in questo modo è elegantissimo. Il *producer* impacchetta il dato e fa una `send`; il *consumer* fa una `receive` e lo estrae. Tutta la complessità dei semafori visti in precedenza qui è gestita "dietro le quinte" dal sistema di messaggistica del sistema operativo!